

APAPO PLATFORM

Domain Operating Systems on Hypergrid

Practical Usage Guide · Domain OS Specification · Integration Reference

Document	Domain OS Practical Usage & General Specification
Version	1.0 · March 2026
Systems	Apapo · Hypergrid NDSS · Kogi (IW-OS) · Ume (B-OS) · Qala (SF-OS)
Status	Authoritative Reference — Active Design
Classification	Confidential — Internal Use Only

Overview

This document does two things. First, it shows how the three Apapo domain operating systems — Kogi (Independent Worker OS), Ume (Business OS), and Qala (Solution Factory OS) — use Hypergrid and Apapo in practice, walking through every major interaction from Grid bootstrap to AI computation to cross-system federation. Second, it abstracts those three concrete implementations into a general-purpose Domain Operating System specification that any future domain OS built on Hypergrid must satisfy.

Hypergrid is the universal N-dimensional distributed spreadsheet substrate. Apapo is the complete integrated platform formed by Hypergrid plus the three domain systems running on it. Every entity in the platform — a freelancer's project, an organization's legal record, a pharmaceutical solution under FDA review — is a HyperRow in a Hypercube, versioned by the same EventLog, synchronized by the same CRDT engine, and reasoned over by the same AI framework.

Core insight: The domain system is not the storage layer. The domain system is a typed lens over Hypergrid. Every domain entity is a HyperRow. Every domain field is a HyperCell. Every domain relationship is a Hypergraph edge. This separation of concerns — universal substrate, typed domain layer — is what makes the platform federable, extensible, and self-governing.

Part I — The Hypergrid Substrate in Context

Before examining how each domain OS uses Hypergrid, it is essential to understand what Hypergrid provides and why every domain system is built on top of it rather than a conventional database.

1. What Hypergrid Provides

Hypergrid is an N-dimensional distributed spreadsheet system. It generalizes the conventional 2D spreadsheet — one of the most universal information organization tools ever invented — along five axes: dimensionality (N dimensions instead of 2), distribution (multi-node CRDT replication instead of file-based), computation (Tier-1 formula + Tier-2 AI columns), graph (embedded Hypergraph layer), and governance (per-attribute permission tiers and policy engine).

Conventional Spreadsheet	Hypergrid Generalization
Workbook	Grid — root runtime container for all Hypercubes, namespaces, spaces, graph, identity
Sheet (2D)	Hypercube $H=(D_1,D_2,\dots,D_N)$ — N-dimensional projection of the Universal Cell Store
Row	HyperRow — all cells sharing D_1 key. The canonical domain entity.
Column	D_2 PropertyAxis — typed, indexed, per-key CRDT semantics and PermissionTier
Cell	HyperCell — N-dim coordinate carrying typed attribute map + causal metadata
Extra dimensions	$D_3\dots D_N$ — Time, Geography, OrgUnit, Tenant, Scenario axes
Formula	ComputedAttribute — Tier-1 (synchronous) or Tier-2 (async AI) over any cell
Filter	DimSlice — predicate on any combination of N axes
Version history	EventLog — every mutation captured; AS_OF time-travel to any prior state
Collaborative editing	VectorClock + per-attribute CRDT — deterministic concurrent merge
External workbook link	CrossGridLink + ShadowCell — typed edge connecting entities across Grids

2. The Apapo Platform Layer

Apapo is the correct name for the convergent whole formed by Hypergrid and the three domain systems. It is not a product layer — it is the unified platform that answers the question: given a universal spreadsheet substrate, what are the three root-domain operating systems that cover the three fundamental categories of operational existence?

System	Domain	Root Concept	Root Hyperspreadsheet	Core Claim
Kogi	Independent Worker OS	The Portfolio	kogi.portfolio.components	Every entity in a worker's life is a row in one living portfolio spreadsheet

System	Domain	Root Concept	Root Hyperspreadsheet	Core Claim
Ume	Business OS	The Organization	ume.kernel.modules + 42 module cubes	Every entity in an organization is a row in one living organization spreadsheet
Qala	Solution Factory OS	The Solution	qala.solutions	Every entity in a factory is a row in one living solution spreadsheet. Kogi and Ume are themselves Solutions.

3. The Three-Layer Architecture

Every domain system has three layers. Understanding this stack is the prerequisite for understanding how each system uses Hypergrid.

Layer	Component	Responsibility	Example (Kogi)
Layer 3 — Domain OS	PortfolioSystem / UmeKernel / QalaOS	Typed domain API; business logic; domain vocabulary	portfolio_system.create_component(...)
Layer 2 — Domain Store	ComponentStore / OrgModuleStore / SolutionStore	Serialize/deserialize domain entities to/from HyperCells	ComponentStore::write(&component) → batch write_cell calls
Layer 1 — Hypergrid	Grid + Hypercubes + Hypergraph + EventLog	Universal storage, CRDT, versioning, federation, AI writeback	grid.write_cell(cube_id, coord, attr, value, actor)

The domain OS never exposes HyperCell, DimCoordinate, or DimKey to its callers. Users of Kogi interact with PortfolioComponents. Users of Ume interact with OrgModules. Users of Qala interact with Solutions. The HyperRow abstraction is entirely internal to Layer 2.

Part II — How a Domain OS Uses Hypergrid in Practice

This section walks through every significant interaction between a domain OS and Hypergrid, using examples drawn directly from Kogi, Ume, and Qala. The patterns are universal — they apply to any domain OS built on Hypergrid.

4. Grid Bootstrap and Domain Registration

When any domain OS starts, it instantiates a Grid and calls its DomainStore bootstrap method. Bootstrap is idempotent — re-registering existing cubes or attribute keys is a no-op. This is the moment when the domain vocabulary is mapped onto Hypergrid's universal schema.

```
// Every domain OS follows this initialization patternlet config = GridConfig {  grid_id:
Uuid::new_v4(),  node_id:      "us-east-1:node-01",  domain_system: DomainSystem::Kogi,
// or Ume / Qala / Standalone  storage: StorageConfig {      primary:
env!("POSTGRES_URL"),          redis:      env!("REDIS_URL"),      kafka:
env!("KAFKA_BROKERS"),          s3_bucket: Some("hypergrid-events"),  },  ai: AiConfig {
mode: AiMode::OpenAiCompatible { ... }, worker_pool_size: 32 },  federation:
FederationConfig { enabled: true, heartbeat_secs: 30 },  ..Default::default();let mut grid
= Grid::new(config).await?; // Kogi registers 8 Hypercubes; Ume registers 44; Qala registers
12KogiDomainStore::bootstrap(&mut grid).await?; // Grid.status → Ready
```

Bootstrap follows a seven-step initialization sequence: config validation, storage connectivity, plugin registry loading, Hypercube registration, EventLog recovery (replaying any unapplied CRDT operations from prior crashes), async warm-up (namespace cache, federation handshake, AI engine health-check), and finally the Ready signal that allows API traffic.

5. Hypercube Schema Design

Each domain OS defines its Hypercubes during bootstrap. The primary Hypercube always follows the N=2 Hybrid encoding pattern (D₁=EntityAxis for entity IDs, D₂=PropertyAxis for field names). Supporting cubes extend to N=3 or N=4 with Time, Category, or Geo axes.

Domain OS	Cube Name	Dimensions	Purpose
Kogi	kogi.portfolio.components	N=2: Entity × Property	Primary cube — all portfolio items and containers
Kogi	kogi.portfolio.kpis	N=3: Entity × Metric × Time	Time-series health, risk, alignment scores per period
Kogi	kogi.portfolio.finances	N=3: Account × Field × Period	Income, expense, budget records with time slicing
Ume	ume.kernel.modules	N=2: Module × Property	Module registry — every organizational function
Ume	ume.chombo.entities	N=3: Entity × Field × Jurisdiction	Legal entities with per-jurisdiction compliance data
Ume	ume.soko.campaigns	N=4: Campaign × KPI × Period × Segment	Marketing analytics with full 4-axis slicing
Qala	qala.solutions	N=2: Solution × Property	Primary cube — all solutions, products, services
Qala	qala.sdes	N=3: SDE × Field × Version	Software Delivery Environments with version axis for AS_OF rollback
Qala	qala.solutions.metrics	N=4: Solution × Metric × Period × Env	Multi-dimensional quality measurement

Every attribute key registered in a Hypercube must declare an explicit CRDT semantics and a write PermissionTier. There are no defaults — every field requires a justified choice.

```
// Pattern shared by all three domain systems
cube.attr_registry.register_batch(vec![ // LWW
  (Last-Write-Wins) - scalar fields where latest state is truth  lww_text("name",
  PermissionTier::Editor),  lww_json("status",  PermissionTier::Editor),  // → Lattice
  in v2  lww_json("visibility",  PermissionTier::Owner),  // OR-Set - set-valued fields
  where concurrent adds both survive  orset_json("owners",  PermissionTier::Owner),
  orset_json("tags",  PermissionTier::Contributor),  orset_json("children",
  PermissionTier::Editor),  // Kogi: child component IDs  orset_json("dependencies",
  PermissionTier::Editor),  // Kogi / Qala: dep edges  // Lattice - lifecycle state that can
  only advance (never regress w/o Admin)  lattice_json("lifecycle_state", MyLifecycleLattice,
  PermissionTier::Manager),  // PNCounter - transactional counter (can go up or down, never
  negative)  pn_number("budget_spent",  PermissionTier::Editor),  // MaxRegister -
  version/sequence that always takes the maximum  max_text("version",
  PermissionTier::System),  // System-only (Tier 8) - AI-computed signals
  ai_attr("health_score",  HEALTH_ENGINE_ID),  ai_attr("risk_score",  RISK_ENGINE_ID),
  ai_attr("anomaly_flag",  ANOMALY_ENGINE_ID),  // Tier-1 formula (synchronous, always
  fresh, never stored)  formula("budget_remaining", Expr::Sub("budget_allocated",
  "budget_spent")),)]?;
```

6. Writing Domain Entities

When a domain OS creates or updates an entity, it translates the typed domain struct into a batch of HyperCell writes via its DomainStore codec. Every field in the domain struct becomes a cell in the Hypercube. The batch write is one PostgreSQL transaction, one EventLog batch entry, and one AI invalidation notification per affected plugin.

The write path is identical for all three domain systems. The nine-step sequence is: permission check, policy engine evaluation, CRDT operation construction, local in-memory merge, PostgreSQL UPSERT, EventLog append, CrdtLog append (for federation), async fan-out (AI invalidation, namespace cache, search index, federation broadcast, WebSocket push).

```
// DomainStore::write - universal pattern // Kogi: ComponentStore::write | Ume:
ModuleStore::write | Qala: SolutionStore::writepub fn write(grid: &mut Grid, cube_id:
CubeId, entity: &MyEntity, actor: &str) -> DomainResult<()>{ let id =
entity.metadata.id; grid.write_cell_batch(cube_id, vec![ // System fields (same for
every domain OS)  (coord(id, "namespace_path"),
Text(entity.metadata.namespace_path.to_string()), actor),  (coord(id, "created_at"),
DateTime(entity.metadata.created_at), actor),  (coord(id, "updated_at"),
DateTime(entity.metadata.updated_at), actor),  (coord(id, "last_actor"),
Text(actor.to_string()), actor),  (coord(id, "vector_clock"),
as_json(&entity.metadata.vector_clock), actor),  (coord(id, "version"),
Text(entity.metadata.version.clone()), actor),  (coord(id, "owners"),
as_json(&entity.metadata.owners), actor),  (coord(id, "tags"),
as_json(&entity.metadata.tags), actor),  (coord(id, "policy_ids"),
as_json(&entity.metadata.policy_ids), actor),  (coord(id, "visibility"),
as_json(&entity.data.visibility), actor), // Domain-specific fields  (coord(id,
"name"), Text(entity.data.name.clone()), actor),  (coord(id, "status"),
as_json(&entity.data.status), actor), // ... all other domain fields ], actor)?;
Ok(())}
```

7. Reading Domain Entities

Reading follows a Redis → computed-attr → PostgreSQL path. Every read applies a VisibilityMask so callers only see fields they are permitted to see. The DomainStore read method takes the raw HyperCells and reconstructs the typed domain struct.

```
pub fn read(grid: &Grid, cube_id: CubeId, id: EntityId) -> DomainResult<Option<MyEntity>> {
// get_row returns all HyperCells for this D; key, visibility-filtered let cells =
grid.get_row(cube_id, &DimKey::uuid(id)); if cells.is_empty() { return Ok(None); } //
Build a field lookup map let fields: HashMap<&str, &TypedAttrValue> = cells.iter()
.filter_map(|c| Some((c.coord.d2_str(), c.value()?))) .collect(); // Reconstruct
typed domain entity from raw cells Ok(Some(MyEntity { metadata: EntityMetadata {
id, owners: fields.from_json("owners"), tags:
fields.from_json("tags"), created_at: fields.from_json("created_at"),
```

```
vector_clock: fields.from_json("vector_clock"),          version:
fields.text("version"),          ...          },          data: EntityData {          name:
fields.text("name"),          status:          fields.from_json("status"),
health_score: fields.opt_f64("health_score"),          // AI-computed          ...          },
}}}}
```

8. CRDT Semantics in Practice

Every domain OS relies on CRDT semantics to resolve concurrent writes without coordination. The choice of CRDT for each field is not arbitrary — it encodes the merge policy that the domain requires.

CRDT Type	Domain Usage	Merge Rule	Example Fields
LastWriteWins (LWW)	Any scalar field where latest state is truth	Higher VectorClock wins; tiebreak by NodeId	name, description, due_date, status (v1)
OR-Set	Any set-valued field where concurrent adds must both survive	All concurrent adds survive; removes are tagged and only remove their exact entry	owners, tags, children, dependencies, team_members, conditions
Lattice	Lifecycle states that must only advance	join(a, b) = least upper bound in partial order; invalid transitions rejected	lifecycle_state, ccr_status, solution_lifecycle, module_state
PNCounter	Transactional accumulators that can go up or down	Sum of (positive ops – negative ops) per node, merged across nodes	budget_spent, hours_logged, units_consumed
GrowOnly Counter	Monotonically increasing metrics	Sum of all increments; decrement is an error	view_count, follower_count, restart_count
MaxRegister	Version/sequence numbers that never go backward	max(a, b) always wins	version, schema_version, sprint_number
AppendLog	Audit trails and comment threads	All appends from all nodes survive; causally ordered	activity_log, comment_thread, decision_log
DeepMergeJson	Configuration blobs requiring partial-key updates	LWW per leaf JSON key; non-overlapping keys always survive	plugin_configs, feature_flags, governance_config

A critical detail: OR-Set removes use a unique tag per add. When Alice adds 'engineering' and Bob concurrently removes 'engineering', the remove only affects Alice's specific tagged add. If Carol also added 'engineering' concurrently, her add survives. This is the correct semantics for shared ownership lists, tag sets, and dependency registries in domain systems.

9. HyperQL — Querying Domain Data

HyperQL is the query language that all three domain systems use to express complex queries across their Hypercubes. It extends SQL with N-dimensional cell references, graph traversal, time-travel, cross-grid federation, and dimension folding.

9.1 Basic Domain Queries

```
-- Kogi: find all active projects for a workerSELECT cell[D1, "name"].value AS name,
cell[D1, "status"].value AS status,          cell[D1, "health_score"].value AS health_score,
cell[D1, "budget_remaining"].value AS budget_remainingFROM kogi.portfolio.componentsWHERE
cell[D1, "status"].value = 'Active' AND cell[D1, "item_type"].value = 'Project'ORDER BY
cell[D1, "health_score"].value DESC;-- Ume: list all running modules with their healthSELECT
cell[D1, "name"].value, cell[D1, "lifecycle_state"].value,          cell[D1,
"health_score"].valueFROM ume.kernel.modulesWHERE cell[D1, "lifecycle_state"].value =
'Running';-- Qala: solutions by maturity stageSELECT cell[D1, "name"].value, cell[D1,
```

```
"maturity_stage"].value, cell[D1, "quality_score"].value, cell[D1,
"defect_density"].valueFROM qala.solutionsWHERE cell[D1, "maturity_stage"].value IN ('"TEST"',
'"CM"')ORDER BY cell[D1, "quality_score"].value DESC;
```

9.2 Time-Travel Queries (AS_OF)

```
-- Kogi: how many active projects did a worker have before a major incident?SELECT COUNT(*) AS
active_project_countFROM kogi.portfolio.components AS_OF '2026-03-15T14:22:00Z'WHERE cell[D1,
"status"].value = '"Active"' AND cell[D1, "item_type"].value = '"Project"';-- Qala: state of
an SDE at a specific deployment versionSELECT cell[D1, "name"].value, cell[D1,
"status"].value, cell[D1, "test_pass_rate"].valueFROM qala.sdes AS_OF
'2026-02-01T00:00:00Z'WHERE cell[D1, "solution_id"].value = '"<solution_uuid>"';
```

9.3 Dimension Folding (N=3+ Cubes)

```
-- Kogi: health score trend over last 4 quartersSELECT D1.entity_id AS component_id,
cell[D1, "name"].value AS name, FOLD D3 WITH AVG AS trendFROM kogi.portfolio.kpisWHERE
D2.metric_name = 'health_score' AND D3.period IN
('2025-Q1','2025-Q2','2025-Q3','2026-Q1')ORDER BY trend DESC;-- Ume: Chombo compliance status
by jurisdictionSELECT D1.entity_id AS entity_id, cell[D1, "name"].value AS entity_name,
EXPAND D3 AS COLUMNS (LAST) -- one column per jurisdictionFROM ume.chombo.entitiesWHERE
cell[D1, "entity_type"].value = '"LegalEntity"';
```

9.4 Graph Traversal

```
-- Kogi: find all transitive dependents of a componentTRAVERSE GRAPH FROM '<component_uuid>'
EDGE_TYPE = Dependency DIRECTION = Inbound MAX_DEPTH = 10 FILTER WHERE cell[D1,
"status"].value != '"Archived"';-- Cross-system: all Kogi workers employed by a Ume
organizationSELECT ume_unit.org_unit_id, cell[ume_unit.org_unit_id, "name"].value AS
unit_name, COUNT(kogi_worker.worker_id) AS contractor_countFROM ume.admin.org_units AS
ume_unitTRAVERSE GRAPH FROM ume_unit EDGE_TYPE = Employs TARGET_GRID = 'kogi://' AS
kogi_workerGROUP BY ume_unit.org_unit_id ORDER BY contractor_count DESC;
```

10. Views as Named DimSlices

Every sheet in a domain OS is a HypercubeView — a saved DimSlice with a declared set of visible attribute keys, a default sort, a default group, and a render mode. The same underlying Hypercube data backs all views simultaneously. Views are not separate tables; they are query configurations stored as HyperRows in a metadata cube.

Domain OS	View Count	Example Views
Kogi	35 sheets	Projects Sheet, Finances Sheet, Benefits Sheet, Gigs Sheet, Kanban Board, Gantt Chart, Network Graph, Health Dashboard, Income Tracker
Ume	42+ views (one per module)	HR Dashboard, Legal Entity Registry, OKR Tracker, Finance Ledger, Campaign Analytics, Risk Matrix, Compliance Calendar
Qala	12+ views	Solutions Registry, SDE Pipeline View, CCR Queue, Release Calendar, Quality Dashboard, Contributor Attribution Grid

```
// Registering a view - identical pattern in all three domain
systemsgrid.register_view(HypercubeView { id: Uuid::new_v4(), cube_id, name:
"Projects Sheet".into(), filter: DimSlice::where_eq("item_type", json_str("Project")),
visible_attrs: vec!["name","status","health_score","due_date","budget_remaining",
"risk_score","owners","tags"], default_sort: SortSpec::desc("health_score"),
default_group: Some(GroupSpec::by("status")), render_mode: RenderMode::Table,})?;
```

11. The AI Computation Pipeline

Every domain OS has an AI engine layer that subscribes to the Hypergrid EventLog via Kafka and computes domain-specific signals (health scores, risk assessments, anomaly flags, income projections) as Tier-2

computed attributes. These signals are written back into the Hypercubes via the WritebackService as TypedAttrValue::AiSignal cells, with a confidence score and staleness marker.

The lifecycle is: cell mutation triggers on_cell_mutation() for all registered HypercubePlugin implementations, which push computation requests onto the AI compute queue. The domain engine picks up the request, runs its model, and posts the result to WritebackService over gRPC. WritebackService validates the PermissionTier::System claim and writes the AiSignal cell.

```
// Tier-2 AI plugin implementation - same interface for all three domain systemsimpl
HypercubePlugin for KogiHealthScoreEngine {    fn id(&self) -> PluginId {
HEALTH_SCORE_ENGINE_ID }    // Declares which mutations should trigger a recompute    fn
on_cell_mutation(&self, cube_id: CubeId, coord: &DimCoordinate)    -> Vec<AttributeKey>
{    // Recompute health_score when any of these fields change    if cube_id ==
PORTFOLIO_CUBE {    match coord.d2_str().as_deref() {
Some("status"|"progress_pct"|"budget_spent"|"risk_score") =>
vec!["health_score".into()],    _ => vec![],    }    } else { vec![] }
}    // Compute the AI signal from current entity state    async fn compute(&self, ctx:
AIComputeContext) -> AIComputeResult {    let row = ctx.grid.get_row(ctx.cube_id,
&ctx.coord.d1_key()).await?;    let score = self.model.evaluate(&row).await?; // domain
model logic    Ok(AIComputeResult {    attr_key:    "health_score".into(),
value:    TypedAttrValue::AiSignal {    value:
Box::new(TypedAttrValue::Number(score)),    model_id:    self.model_id.clone(),
computed_at: Utc::now(),    confidence:    0.87,    stale_at:
Some(Utc::now() + Duration::hours(1)),    },    cube_id: ctx.cube_id,
coord:    ctx.coord,    })    }}
```

Part III — Kogi in Practice

Independent Worker OS · Portfolio as Root Concept

12. Kogi Identity Card

Attribute	Value
Domain OS Name	Kogi (IW-OS — Independent Worker Operating System)
Root Domain Concept	The Portfolio
Primary Hyperspreadsheet	kogi.portfolio.components
Root Entity Type	PortfolioComponent (Item Container)
Domain Prefix	kogi://
Hypercube Count	8 Hypercubes (N=2 to N=3)
AI Engine	kogi-engine (Scala 3) — 12 sub-engines
AI Assistant	Oba — NL-to-HyperQL + autonomous agent modes
CrossGridLinks Supported	Kogi↔Ume (employment) · Kogi↔Qala (contribution) · Kogi↔Kogi (KLNK economic graph)

13. The Portfolio as Root Concept

In Kogi, every entity in a worker's operational, creative, and financial life — programs, projects, tasks, resources, assets, gigs, benefits, finances, relationships, governance events — is a single typed row in the kogi.portfolio.components Hypercube. This is the Portfolio: not a container for projects, but the unified living record of the worker's entire existence on the platform.

A PortfolioComponent is either an Item (Project, Task, Gig, Resource, Asset, BenefitAccount, Contract, Job, Campaign, Grant, Investment, Profile) or a Container (Binder, Book, Record, Folder, Registry, Archive). Both types share a universal row schema — the same 250+ columns — but with different active column groups determined by the type discriminant.

14. Kogi Hypercube Usage

Hypercube	Dimensions	Purpose	Key CRDT Choices
kogi.portfolio.components	N=2: Entity × Property	All portfolio items and containers	OR-Set: owners, tags, children, dependencies; Lattice: ComponentStatus; PNCounter: budget_spent
kogi.portfolio.kpis	N=3: Entity × Metric × Period	Time-series KPI tracking per quarter/sprint	LWW: score values; MaxRegister: period sequence
kogi.portfolio.finances	N=3: Account × Field × Period	Income, expense, tax records sliced by period	PNCounter: balance; LWW: category, notes
kogi.portfolio.benefits	N=2: Benefit × Property	Portable benefit coverage records	LWW: coverage status; OR-Set: coverage_sources
kogi.portfolio.actions	N=2: Action × Property	Pending and completed governance actions	Lattice: action_status
kogi.portfolio.approvals	N=2: Approval × Property	Approval requests from other systems	Lattice: approval_status; OR-Set: approvers

Hypercube	Dimensions	Purpose	Key CRDT Choices
kogi.portfolio.links	N=2: Link × Property	CrossGridLink records and attribution weights	LWW: link metadata; OR-Set: mirrored_attrs
kogi.portfolio.snapshots	N=2: Snapshot × Property	Point-in-time portfolio snapshots for audit	LWW: snapshot data; AppendLog: restore_events

15. kogi-engine AI Integration

kogi-engine is a Scala 3 service that consumes the Kogi Grid's EventLog via Kafka and runs 12 sub-engines concurrently. Each engine is a HypercubePlugin registered with the Grid that declares which cell mutations trigger a recompute and writes results back via the WritebackService.

Sub-Engine	Output Attribute	Target Cube	Trigger Fields
KogiHealthScoreEngine	health_score	kogi.portfolio.kpis	status, progress_pct, budget_spent, risk_score
KogiRiskEngine	risk_score	kogi.portfolio.kpis	dependencies, budget_utilization, schedule_variance
KogiAlignmentEngine	alignment_score	kogi.portfolio.kpis	okr_refs, tags, item_type, parent_id
KogiAnomalyEngine	anomaly_flag	kogi.portfolio.components	Any mutation in health_score, budget_spent, velocity
KogiMatchEngine	match_score	kogi.portfolio.links	New CrossGridLink or mutation to linked entity
KogiIncomeProjectionEngine	income_projection_90d	kogi.portfolio.finances	gig earnings, contract status, rate changes
KogiBudgetEngine	budget_burn_rate	kogi.portfolio.finances	budget_spent counter increments
KogiNarrativeEngine	narrative_summary	kogi.portfolio.components	name, status, health_score, recent activity events
KogiScheduleRiskEngine	predicted_completion_date	kogi.portfolio.components	velocity, story_points, sprint_ref changes
KogiCoverageGapEngine	coverage_gap_flags	kogi.portfolio.benefits	benefit type, balance, provider mutations

16. Oba — The Kogi AI Assistant

Oba is Kogi's AI assistant. It operates over HyperQL — Oba translates natural-language requests into HyperQL queries, executes them against the Kogi Grid, and can write results back as EventLog annotations or cell mutations. Oba operates in nine modes:

Mode	Description	Example
Reactive	Answers queries on demand	"Show me all projects over budget" → HyperQL + formatted result
Proactive	Pushes anomaly alerts and health briefings	Notifies worker when a project's health score drops below threshold
ColumnCompletion	Suggests values for incomplete fields	Suggests due_date based on similar past projects

Mode	Description	Example
SmartFilter	Constructs view filters from NL descriptions	"Show me risky projects" → DimSlice(risk_score > 0.7, status = Active)
FormulaSuggestion	Proposes Tier-1 formula columns	"I want to see remaining capacity" → FormulaExpr for capacity – allocated
BatchAction	Applies bulk mutations from NL descriptions	"Archive all completed projects older than 6 months" → batch Lattice transitions
HealthBriefing	Generates portfolio health narrative from AI signals	Weekly health report combining all 12 engine signals
NarrativeGeneration	Writes narrative_summary cells for components	Produces human-readable project status from raw HyperRow data
Autonomous	Plans and executes multi-step workflows	"Move this project to the next sprint and notify all collaborators"

17. KLNK — The Kogi Link Network

KLNK (Kogi Link Network) is the economic graph of interconnected Portfolio nodes across the entire Kogi platform. Every worker's Kogi Grid is a node in this graph. CrossGridLink edges with ShadowCells represent connections: employment, investment, collaboration, cooperative membership, marketplace transactions.

KLNK edges are CrossGridLink edges with domain-specific semantics. Every KLNK edge requires consent, provisions a ShadowCell, and establishes a real-time sync channel. The ShadowCell in the requesting node mirrors the permitted columns from the connected node.

KLNK Edge Type	Direction	Consent	Mirrored Columns	Example Use
Employs	Org→Worker	Worker consent required	work_record fields (tasks, availability)	Ume organization employing a Kogi independent worker
Collaborates	Bidirectional	Both parties consent	contribution fields, attribution_weight	Two workers co-authoring a project
InvestedIn	Investor→Target	Target consent required	portfolio summary, public health_score	Angel investor tracking a worker's portfolio
OrgMembership	Org→Member	Member consent	name, status, health_score (public columns)	Worker joining a cooperative or collective
Contracted	Client→Worker	Worker consent	contract status, deliverables (scoped)	Freelance contract relationship
MarketplaceTransaction	Buyer→Seller	Via transaction acceptance	transaction record only	Kogi marketplace purchase

18. Multi-Identity (KPID)

Kogi supports a sovereign entity owning multiple distinct identities, each with its own ProfilePartition, VisibilityMask, and isolation level. A worker might maintain a 'dev' identity (showing code projects), a 'design' identity (showing creative work), and a 'consulting' identity (showing client-facing work). All identities share the same root Hypercube but with different row filtering and column visibility.

Three isolation levels exist: None (default, cross-identity reads permitted), SoftIsolated (cross-identity reads require explicit permission), and HardIsolated (complete data separation — even authenticated reads from the same Sovereign Entity cannot see HardIsolated partition data without the partition's own authentication).

Part IV — Ume in Practice

Business OS · Organization as Root Concept

19. Ume Identity Card

Attribute	Value
Domain OS Name	Ume (B-OS — Business Operating System)
Root Domain Concept	The Organization
Primary Hyperspreadsheet	ume.kernel.modules (registry) + 42 module Hypercubes
Root Entity Type	OrgModule (any organizational function)
Domain Prefix	ume://
Hypercube Count	44 (1 kernel + 42 modules + 1 template library)
Dimension Range	N=2 to N=4
AI Engine	ume-engine (Go + Python) — 8 core capabilities
CrossGridLinks Supported	Ume↔Kogi (employment, membership) · Ume↔Qala (product catalog, OKR alignment)

20. The Organization as Root Concept

In Ume, the Organization is not one module among many. It is the system. The 42 module Hypercubes collectively represent every organizational function — and the organization's existence in the system is the sum of all those rows across all those cubes. A new organization is bootstrapped by instantiating its module registry (ume.kernel.modules) and activating the modules relevant to its domain.

The Module Registry cube is the organization chart at the functional level. Each row is a registered organizational function (Finance, HR, Legal, Marketing, Engineering, etc.). The 42 module cubes each store the actual operational records for that function. Modules 41 and 42 are extensible namespace slots for custom organization-specific modules.

21. Ume 42 Module Hypercubes — Selected Examples

Module	Cube Name	Dimensions	Purpose
01 Admin	ume.admin.org_units	N=2	Organization hierarchy and unit records
02 HR	ume.hr.employees	N=3: Employee × Field × Period	Employee records with temporal performance tracking
07 Finance	ume.finance.ledger	N=3: Entry × Field × Period	Financial ledger with period-sliced analysis
11 Sales	ume.sales.opportunities	N=3: Opp × Field × Stage	Sales pipeline with stage progression
13 Chombo	ume.chombo.entities	N=3: Entity × Field × Jurisdiction	Legal entities with per-jurisdiction compliance data. The N=3 axis enables simultaneous tracking of compliance status across all relevant jurisdictions.
22 Soko	ume.soko.campaigns	N=4: Campaign × KPI × Period × Segment	Marketing analytics. The N=4 axis enables slicing by time period AND audience segment simultaneously — impossible in conventional 2D tools.

Module	Cube Name	Dimensions	Purpose
26 GRC	ume.grc.risks	N=2	Risk register with AI-computed correlation analysis
30 Strategy	ume.strategy.okrs	N=3: OKR × Field × Quarter	Objectives and key results with quarterly tracking

22. Ume Kernel-to-Hypergrid Mapping

UmeKernel is the root orchestrator for the Ume domain OS. It maps each of its 12 kernel services onto Hypergrid infrastructure components. The mapping is explicit and documented — no kernel service touches storage directly; all storage operations go through the Hypergrid layer.

Kernel Service	Hypergrid Component	Key Operations
ModuleRegistry	ume.kernel.modules Hypercube	register_module(), get_module_health(), list_active_modules()
GovernanceEngine	PolicyEngine + Lattice CRDT	evaluate_policy(), require_approval(), validate_lifecycle_transition()
AuditEngine	EventLog + AppendLog CRDT	All mutations produce immutable EventLog entries; AS_OF for audit queries
IdentityManager	HG-ID IdentityStore	create_entity(), assign_role(), enforce_rbac()
DataEngine	HG-CELL + HG-GRAPH + HG-QL	write_record(), query(), traverse_org_graph()
AIEngine	HG-AI + WritebackService + Kafka	All AI signals via ume-engine plugin + WritebackService gRPC
FederationManager	HG-FED CrossGridLink protocol	link_to_kogi(), link_to_qala(), manage_shadow_cells()
SearchService	HG-NS + Meilisearch	Full-text search across all 42 module cubes

23. Chombo N=3 in Detail

Chombo (Module 13, Compliance & Regulatory) demonstrates why Hypergrid's N-dimensional model is architecturally necessary. A legal entity operating across multiple jurisdictions has compliance obligations that are simultaneously (a) entity-specific, (b) field-specific, and (c) jurisdiction-specific. A conventional 2D table cannot express this without schema explosion.

In Ume, `ume.chombo.entities` uses $N=3$: D_1 =EntityAxis (legal entity), D_2 =PropertyAxis (field name), D_3 =CategoryAxis (jurisdiction code, e.g. 'US-CA', 'UK', 'EU-GDPR'). A single HyperQL query can return one column per jurisdiction, or filter to show only out-of-compliance jurisdictions, or compute the entity's worst compliance posture across all jurisdictions simultaneously.

```
-- Chombo: compliance posture for all entities, all jurisdictions
SELECT D1.entity_id, cell[D1, "name"].value AS entity_name,
       EXPAND D3 AS COLUMNS (LAST) -- one column per jurisdiction
FROM ume.chombo.entities WHERE cell[D1, "entity_type"].value = "LegalEntity";
-- Chombo: entities with any out-of-compliance jurisdiction
SELECT D1.entity_id, cell[D1, "name"].value,
       cell[D1, "compliance_prediction"].value AS ai_prediction
FROM ume.chombo.entities WHERE cell[D1, D2, D3, "compliance_status"].value = "Non-Compliant" AND
D3.jurisdiction IS NOT NULL;
```

24. ume-engine AI Integration

Capability	Output Attribute	Target Cube	Description
UmeModuleHealthEngine	health_score	ume.kernel.modules	Aggregate health of each organizational module based on its underlying records
UmeChomboAI	compliance_prediction	ume.chombo.entities	AI prediction of compliance status for upcoming regulatory deadlines
UmeStrategyAI	okr_completion_prob	ume.strategy.okrs	Probability of OKR completion based on current key result progress
UmeHRAI	attrition_risk_score	ume.hr.employees	Employee attrition risk based on engagement, tenure, performance signals
UmeFinanceAI	revenue_forecast_q	ume.sales.opportunities	Quarterly revenue forecast from pipeline analysis
UmeRiskAI	correlated_risk_ids	ume.grc.risks	Identifies risk correlations across the GRC risk register
UmeSokoAI	optimization_suggestions	ume.soko.campaigns	Campaign budget optimization suggestions from performance data
UmeAnomalyEngine	anomaly_flag	All module cubes	Cross-module anomaly detection — triggers alerts when metrics diverge from baselines

Part V — Qala in Practice

Solution Factory OS · The Solution as Universal Category

25. Qala Identity Card

Attribute	Value
Domain OS Name	Qala (SF-OS — Solution Factory Operating System)
Root Domain Concept	The Solution (Universal Category)
Primary Hyperspreadsheet	qala.solutions
Root Entity Type	Solution (Application System Good Product Service Platform)
Domain Prefix	qala://
Hypercube Count	12 core + Domain Pack extension cubes
AI Engine	qala-engine (Rust) — 10 AI Agent capabilities
Self-governing	YES — Kogi and Ume are HyperRows in qala.solutions
CrossGridLinks Supported	Qala↔Kogi (contribution attribution) · Qala↔Ume (product catalog, OKR alignment)

26. The Solution as Universal Category

Qala's root concept is the most philosophically powerful aspect of the Apapo platform. The Solution is defined as any purposeful output designed to address a problem, fulfill a goal, or produce an intended outcome. This definition is deliberately universal: it encompasses software applications, physical products, ongoing services, and entire platforms.

The consequence of this universality is that Kogi and Ume are themselves Solutions in Qala's ontology. They are HyperRows in qala.solutions with type=Platform, governed by the Root Factory, subject to the standard solution lifecycle (Draft → InReview → Approved → Active → Deprecated → Retired), and versioned through the Change Control Request (CCR) workflow. The platform governs itself using its own tools.

27. Qala Hypercube Usage

Hypercube	Dimensions	Key Features
qala.solutions	N=2: Solution × Property	All solution entities; lifecycle_state as Lattice CRDT; Domain Pack extensions add attribute keys at runtime
qala.sdes	N=3: SDE × Field × Version	Software Delivery Environments with version axis enabling AS_OF rollback to any prior SDE configuration
qala.ccrs	N=2: CCR × Property	Change Control Requests; status as Lattice CRDT (Draft→InReview→Approved→Rejected); OR-Set on approver_ids
qala.releases	N=2: Release × Property	Release records linking CCR approval to deployed artifacts; immutable after creation
qala.solutions.metrics	N=4: Solution × Metric × Period × Environment	Multi-dimensional quality measurement enabling slice by time AND environment (dev/test/prod) simultaneously
qala.domain_packs	N=2: Pack × Property	Domain Pack registry — each pack extends qala.solutions with domain-specific attribute keys and governance rules

Hypercube	Dimensions	Key Features
qala.factories	N=2: Factory × Property	Solution Factory instances, each with its own child solution roster and Domain Pack configuration

28. The CCR Workflow on Hypergrid

Every significant change to a Solution — a new version, a lifecycle transition, an architecture change — must go through a Change Control Request. The CCR workflow is fully modeled in Hypergrid: each CCR is a HyperRow in `qala.ccrs`, the status field uses a Lattice CRDT to enforce forward-only progression, and the approver set uses an OR-Set CRDT so concurrent additions of approvers both survive.

```
// CCR lifecycle on Hypergrid// 1. Create the CCRqala.create_ccr(CreateCcrRequest {
solution_id:   KOGI_PLATFORM UUID,   change_type:   ChangeType::FeatureRelease,
description:   "Kogi v1.2.0 - KLNK federation improvements",   impact_level:
ImpactLevel::Medium,   submitter:   actor,})?;// → new HyperRow in qala.ccrs// → Lattice:
status = Draft// → AI Agent queued to compute risk assessment// 2. Submit for review (Lattice
transition: Draft → InReview)qala_os.transition_ccr_status(ccr_id, CcrStatus::InReview,
actor)?;// 3. Add approvers (OR-Set: concurrent adds both
survive)grid.apply_crdt_op(CrdtOperation::AddToSet {   coord:   coord(ccr_id,
"approver_ids"),   element:   TypedAttrValue::User(approver_id),   unique_tag:
Uuid::new_v4(),   actor,})?;// 4. When all approvers sign: Lattice transition → Approved//
GovernanceEngine policy: require ALL approver signatures// Policy blocks transition if any
approver has not signed// 5. Release record created on approval// 6. Solution HyperRow
lifecycle_state advanced by Lattice CRDT
```

29. Domain Packs as Hypergrid Plugins

Domain Packs are the primary extension mechanism in Qala. A Domain Pack adds domain-specific attribute keys to `qala.solutions`, registers new governance rules in the PolicyEngine, installs AI sub-engine plugins for domain signals, and can add new view configurations. From Hypergrid's perspective, a Domain Pack is a HypercubePlugin implementation that declares new attribute key definitions to be registered with `qala.solutions` at bootstrap.

Domain Pack	Added Attribute Keys (examples)	AI Sub-Engine
Software Pack	build_system, test_framework, security_scan_status, sast_score, sbom_ref	SoftwareQualityEngine: defect_density, code_coverage_pct
Pharmaceutical Pack	anda_number, fda_submission_status, gmp_audit_date, stability_data_ref	PharmaRegulatoryEngine: regulatory_risk_score, submission_prediction
Hardware Pack	bom_version, manufacturing_site, iso_certifications, eol_date	HardwareRiskEngine: supply_chain_risk_score
Financial Instrument Pack	isin, asset_class, regulatory_classification, prospectus_ref	FinancialRiskEngine: market_risk_score, liquidity_score

30. qala-engine AI Capabilities

Capability	Output Attribute	Description
QalaRiskEngine	risk_score	CCR risk scoring from change type, impact level, and historical CCR data
QalaDriftDetector	drift_status	Detects schema and configuration drift in SDEs versus their baseline
QalaQualityEngine	quality_score	Multi-signal quality score from test pass rate, defect density, code coverage

Capability	Output Attribute	Description
QalaComplianceEngine	compliance_prediction	Domain Pack-specific compliance prediction (Software Pack: SAST pass probability)
QalaReleaseScheduler	predicted_release_date	Release date prediction from SDE velocity and CCR queue depth
QalaAnomalyDetector	anomaly_flag	Anomaly detection across all solution metrics
QalaAttributionEngine	attribution_weights	Contributor attribution weights from commit graph, review history, design docs
QalaChurnPredictor	churn_risk_score	Solution abandonment risk from activity signals, contributor count, issue velocity

Part VI — Cross-System Integration in Practice

The three domain systems are independent Grids, each with its own storage, EventLog, and CRDT namespace. Cross-system connections are modeled as CrossGridLink edges in the Hypergraph, using the ShadowCell protocol for data mirroring and the federation coordinator for routing.

31. Kogi ↔ Ume Integration

Integration Scenario	Edge Type	Consent	Data Flow	Practical Usage
Employment	Employs (Ume org → Kogi worker)	Worker consent required	work_record fields mirror to Ume ShadowCell; read-only from Ume side	An Ume organization sees contracted workers' task load and availability without accessing private portfolio data
Cooperative membership	OrgMembership (Ume org → Kogi worker)	Worker consent required	Public portfolio columns (name, status, tags, health_score) mirror to Ume	A cooperative's member roster in Ume reflects real-time portfolio health of each member worker

```
// Kogi ↔ Ume employment link — practical flow// Step 1 (Ume side): Request employment
linkume kernel.create_employment_link(CreateEmploymentLinkRequest { org_id:
ACME_ORG_UUID, kogi_path: "kogi://alice-dev/profile/main/", scope:
EmploymentScope::Contractor,});// → CrossGridLink edge created in Ume Hypergraph// →
ConsentRequest sent to Alice's Kogi Grid// Step 2 (Kogi side): Alice reviews and
acceptskogi.accept_employment_link(link_id, ConsentConfig { mirrored_attrs:
vec!["task_count", "availability_hours", "current_gig_count"], writeback_attrs: vec![], //
Alice grants no write-back update_policy: UpdatePolicy::RealTime,});// Step 3: Automatic —
ShadowCell provisioned in Ume Grid// Step 4: Automatic — Any change to Alice's mirrored
fields// → Kafka shadow.sync topic → Federation Coordinator// → Ume Grid applies delta to
ShadowCell// → Ume EventLog: ShadowCellUpdated { source: kogi://alice-dev/... }// Ume
HyperQL: query workers across the org, using shadow dataSELECT kw.shadow_id,
cell[kw.shadow_id, "task_count"].value AS tasks, cell[kw.shadow_id,
"availability_hours"].value AS capacityFROM ume.admin.shadow_workers AS kwWHERE
cell[kw.shadow_id, "org_id"].value = '<acme_uuid>'ORDER BY capacity DESC;
```

32. Kogi ↔ Qala Integration

When a Kogi worker contributes to a Qala solution, a CrossGridLink edge connects their portfolio item to the solution entity. The link tracks attribution_weight — the worker's proportional contribution to the solution. The Qala side mirrors the solution lifecycle state and version into the worker's portfolio; the Kogi side exposes contribution records to the Qala attribution engine.

33. Ume ↔ Qala Integration

An Ume organization's product catalog links to the Qala solutions that implement those products. Ume OKRs can be associated with Qala SDEs, enabling the strategy module to track delivery velocity against strategic objectives. These integrations require no consent (they are within the same organization's federated deployment) and use Association edges rather than CrossGridLinks requiring the full consent protocol.

34. Cross-System HyperQL

When both Grids are registered as federation peers, HyperQL can span Grid boundaries using graph traversal clauses. The federation coordinator fans out the query to each involved Grid and merges the results.

```
-- Platform health: all three systems in one querySELECT 'kogi' AS system, AVG(cell[D1, D2,
"health_score", D3].value) AS avg_health, COUNT(D1.entity_id) AS active_countFROM
```

```
kogi.portfolio.kpisWHERE cell[D1, "status"].value = '"Active"' AND D2.metric_name =
'health_score'UNION ALLSELECT 'ume', AVG(cell[D1, "health_score"].value),
COUNT(D1.entity_id)FROM ume.kernel.modules WHERE cell[D1, "lifecycle_state"].value =
'"Running"'UNION ALLSELECT 'qala', AVG(cell[D1, "quality_score"].value),
COUNT(D1.entity_id)FROM qala.solutions WHERE cell[D1, "lifecycle_state"].value = '"Active"';--
Contribution chain: solution → workers → their health scoresSELECT q.solution_id,
cell[q.solution_id, "name"].value AS solution_name,          k.worker_id, cell[k.worker_id,
"name"].value AS worker_name,          cell[k.worker_id, D2, "health_score", "2026-Q1"].value AS
health,          link.attribution_weightFROM qala.solutions AS qJOIN kogi.portfolio.links AS link
ON GRAPH_EDGE(q.solution_id, link.component_id, 'CrossGridLink')JOIN kogi.portfolio.components
AS k ON link.source_entity_id = k.entity_idWHERE cell[q.solution_id, "lifecycle_state"].value
= '"Active"'ORDER BY link.attribution_weight DESC;
```

Part VII — The General Domain Operating System Specification

This section defines the complete, abstract specification for any Domain Operating System built on Hypergrid. It is derived by abstracting the three concrete implementations (Kogi, Ume, Qala) into a general contract. Any future domain OS — Healthcare OS, Legal OS, Supply Chain OS, Real Estate OS, Financial Portfolio OS — must satisfy this specification to be a first-class Hypergrid citizen eligible for cross-system federation.

DOMAIN-OS-SPEC-000: This specification is normative. 'MUST' requirements are enforced at platform integration points. 'SHOULD' requirements are strongly recommended. Violations of MUST requirements prevent federation, unified audit access, and platform-level governance.

35. What is a Domain Operating System?

A Domain Operating System (Domain OS) is a software system that provides a complete, opinionated operational environment for entities within a specific domain, built on top of Hypergrid as its universal data substrate.

A Domain OS has five defining characteristics:

Characteristic	Definition	Example (Kogi)
Single root domain concept	One entity type that is the center of gravity for the system. Everything else exists to enrich, relate to, or govern this concept.	The Portfolio. Every entity in the worker's life is subordinate to the Portfolio.
Root Hyperspreadsheet	The primary Hypercube storing root domain entities as HyperRows. All other cubes support or extend this one.	kogi.portfolio.components — the master portfolio spreadsheet
Domain-typed API surface	The system exposes domain vocabulary, not Hypergrid vocabulary. Users see domain entities, not HyperRows.	create_component(), get_health_dashboard(), link_to_organization()
Domain intelligence layer	AI engines computing domain-specific signals as Tier-2 computed attributes.	12 kogi-engine sub-engines producing health scores, risk assessments, income projections
Federated graph	Domain-typed subset of the Hypergraph, with cross-system connections via CrossGridLink.	KLNK: the economic graph of inter-portfolio connections

36. Mandatory Structural Requirements

DOMAIN-OS-SPEC-001: Root Domain Concept

Every Domain OS MUST declare exactly one root domain concept. The root concept MUST be a typed entity stored as a HyperRow in the primary Hypercube. All other entities in the system MUST have a defined relationship to the root concept, expressed as one of: part-of, belongs-to, governs, supports, or derives-from. A domain OS without a clearly identified root concept does not qualify as a Domain OS — it is a collection of disconnected cubes.

DOMAIN-OS-SPEC-002: Primary Hypercube

Every Domain OS MUST register exactly one primary Hypercube named {domain_prefix}.{root_entity_type_plural}. The primary Hypercube MUST use the N=2 Hybrid encoding (D₁=EntityAxis, D₂=PropertyAxis) as the base, with additional dimensions as domain-required. All root domain entities MUST be stored as HyperRows in this cube.

DOMAIN-OS-SPEC-003: DomainStore Codec

Every Domain OS MUST implement a DomainStore codec with four methods:

```
trait DomainStore<Entity, EntityId> { // Register all Hypercubes and attribute key schemas
  (idempotent) fn bootstrap(grid: &mut Grid) -> HypergridResult<>; // Serialize domain
  entity -> HyperCells and write to grid fn write(grid: &mut Grid, cube_id: CubeId, entity:
  &Entity, actor: &str) -> DomainResult<>; // Read HyperCells -> deserialize into
  domain entity (or None if not found) fn read(grid: &Grid, cube_id: CubeId, id: EntityId)
  -> DomainResult<Option<Entity>>; // Check existence without full deserialization fn
  exists(grid: &Grid, cube_id: CubeId, id: EntityId) -> bool; } // RULE: The DomainStore codec
  MUST be the ONLY place where domain entity// structs are serialized to or deserialized from
  HyperCells.// Domain logic MUST NOT access HyperCells directly.
```

DOMAIN-OS-SPEC-004: Root Orchestrator

Every Domain OS MUST have a root orchestrator struct that owns the Grid, exposes all domain operations as typed methods, delegates all storage to the DomainStore codec, emits dual-write events on all mutations, and never exposes HyperCell, DimCoordinate, or DimKey to its callers.

DOMAIN-OS-SPEC-005: CRDT Assignment

Every attribute key registered in every Domain OS Hypercube MUST have an explicit CrdtSemantics assignment. No attribute key may use the default (LWW) without explicit justification. The CRDT Decision Tree (below) MUST be applied to every field:

Question	Yes → CRDT	No → Continue
Is this a set-valued field where concurrent adds from multiple users/nodes should ALL survive?	OR-Set	Is it a lifecycle state?
Is it a lifecycle state that should only advance (never regress without Admin override)?	Lattice	Is it an accumulator?
Is it an accumulator that can both increase and decrease (e.g. budget spent, hours logged)?	PNCounter	Is it always increasing?
Is it always increasing (views, likes, restarts)?	GrowOnly Counter	Is it a version/sequence number?
Is it a version/sequence number that should always take the highest value?	MaxRegister	Is it an audit trail?
Is it an append-only audit trail or comment thread?	AppendLog	Is it a JSON config blob needing partial-key updates?
Is it a JSON configuration blob where concurrent partial-key updates should both survive?	DeepMergeJson	Use LWW

DOMAIN-OS-SPEC-006: Permission Tier Assignment

Every attribute key MUST have an explicit write_permission PermissionTier. AI-computed attributes MUST use PermissionTier::System. System metadata (vector_clock, last_actor, created_at) MUST use PermissionTier::System. No attribute key may use PermissionTier::Viewer (0) for write access.

DOMAIN-OS-SPEC-007: Dual-Write Event Architecture

Every mutating operation in the Domain OS MUST emit two events synchronously before async fan-out: a domain-typed event to the domain event log, and a Hypergrid EventEntry to the shared EventLog as EventKind::Custom with a domain tag in the format {domain_prefix}:{event_kind}.

DOMAIN-OS-SPEC-008: NamespacePath Assignment

Every root domain entity MUST have a NamespacePath assigned at creation, registered in the Grid's NamespaceRegistry. Path format: {domain}://{space_slug}/{entity_type_slug}/{entity_slug}/. Cross-system links MUST use fully-qualified NamespacePaths.

DOMAIN-OS-SPEC-009: HyperQL Passthrough

Every Domain OS root orchestrator MUST expose a hyperql() method that passes HyperQL queries directly to the underlying Grid without modification. This enables AI engines, analytics tools, and advanced users to query the Hypercube substrate directly, bypassing the domain API when needed.

DOMAIN-OS-SPEC-010: Stats Exposure

Every Domain OS root orchestrator MUST expose a stats() method returning both domain-level metrics (entity count, edge count, event count) and Hypergrid substrate metrics (hg_total_cells, hg_event_count, hg_crdt_ops). This enables platform-level monitoring without coupling to domain internals.

37. Mandatory Attribute Keys

Every primary Hypercube in every Domain OS MUST register the following attribute keys with the specified CRDT semantics and permission tiers. These are the platform contract — without them, unified audit, federation, and governance cannot function.

Attribute Key	Type	CRDT	PermissionTier	Purpose
created_at	DateTime	LWW	System	Entity creation timestamp
updated_at	DateTime	LWW	System	Last mutation timestamp — updated on every write
last_actor	Text	LWW	System	Nodeld of last writer — CRDT tiebreak for LWW conflicts
vector_clock	Json	LWW	System	Causal clock snapshot at last write — required for federation merge
version	Text	MaxRegister	Editor	Semantic version string — always takes highest value
status	Json	LWW (→Lattice v2)	Editor	Lifecycle status enum — domain-specific value set
visibility	Json	LWW	Owner	Visibility: Private Protected Public
owners	Json	OR-Set	Owner	Vec<UserId> — primary ownership; OR-Set for concurrent adds
tags	Json	OR-Set	Contributor	Freeform labels — OR-Set for concurrent tag additions
policy_ids	Json	LWW	Admin	Governance policy attachments
namespace_path	Text	LWW	System	NamespacePath URI — required for cross-system linking

38. Mandatory Hypergraph Edge Types

Every Domain OS **MUST** support the following edge types in its domain graph. Additional edge types are domain-specific and optional.

EdgeType	Required	Cycle Detection	Consent	Purpose
Hierarchy	All Domain OSes	Not enforced (parent-child can have diamonds)	No	Parent-child containment for nested entity structures
Dependency	All Domain OSes	Enforced (DFS) — MUST reject cycles	No	Blocking dependency — source cannot complete before target
Association	All Domain OSes	Not enforced	No	Soft lateral reference — no blocking semantics
CrossGridLink	All Domain OSes	N/A (cross-grid)	Yes — required before ShadowCell provisioning	Inter-grid connection — the fundamental link network atom

39. AI Integration Requirements

Every Domain OS **SHOULD** define at least one primary AI signal on its root domain entities. Every AI engine **MUST** implement the HypercubePlugin interface. All AI signals **MUST** be written as TypedAttrValue::AiSignal with a confidence score and staleness marker, via the WritebackService gRPC endpoint. AI signals **MUST** use PermissionTier::System — no human actor may write to an AI-computed field.

Required AI Signal Category	Minimum Definition	Example
Health signal	MUST have at least one aggregate health/vitality score for root entities	health_score (0–100) combining status, progress, and resource utilization
Risk signal	MUST have at least one risk or anomaly flag for root entities	risk_score or anomaly_flag triggered by threshold violations
Lifecycle prediction	SHOULD have at least one predictive signal about future state	predicted_completion_date, compliance_prediction, attrition_risk_score

40. Space and Namespace Design Requirements

Every Domain OS **MUST** define a Space taxonomy declaring which SpaceTypes its entities can exist in. Every entity creation **MUST** result in a Space membership assignment. At minimum, every Domain OS must support: Personal spaces (owner-only governance), Team spaces (manager-approval governance), and Community spaces (member-vote governance with configurable quorum).

Every entity **MUST** be assigned a NamespacePath URI at creation. The namespace scheme **MUST** follow the pattern {domain}://{space_slug}/{entity_type_slug}/{entity_slug}/. Namespace conflicts (two entities with the same path) **MUST** be detected at creation time and rejected.

41. Federation Requirements

Every Domain OS **MUST** be capable of federating with other Domain OS instances. Federation has two forms: same-system multi-node (via CrdtLog delta over Kafka) and cross-system (via CrossGridLink + ShadowCell protocol). A Domain OS that cannot participate in either form of federation is not a first-class Hypergrid citizen.

Federation Requirement	MUST / SHOULD	Implementation
Same-system multi-node CRDT sync	MUST	CrdtLog → Kafka topic crdt.delta.{grid_id} → consumer on peer → apply_crdt_op()
CrossGridLink consent protocol	MUST	Six-phase protocol: request, configure, provision, sync, writeback, lifecycle
ShadowCell real-time sync	MUST	EventLog → Kafka shadow.sync.{grid_id} → Federation Coordinator → peer ShadowCell write
Offline / edge deployment	SHOULD	SQLite backend for offline accumulation; delta sync on reconnect; CRDT convergence guaranteed
Cross-grid HyperQL	SHOULD	Federation coordinator fans out queries to peer grids; results merged in query engine

42. Performance Targets

Every Domain OS built on a standard Hypergrid deployment (8 vCPU, 32 GB RAM, NVMe SSD) **MUST** meet the following performance targets as measured against its primary Hypercube:

Operation	Target p50	Target p99	Notes
Single HyperCell read (cache hit)	<1ms	<5ms	Redis L1 cache
HyperRow read, all fields (cache miss)	<10ms	<40ms	PostgreSQL primary key scan
Batch write, 30 fields	<15ms	<50ms	One PG transaction
DimSlice query, <1K rows	<50ms	<200ms	PG GIN/BTree index
DimFold, N=3, <100K rows	<500ms	<2s	PG window functions
Graph BFS, depth=3	<50ms	<200ms	PG recursive CTE
AS_OF time-travel (hot window)	<100ms	<500ms	EventLog replay
CrossGridLink delta sync	<500ms	<2s	Kafka propagation + ShadowCell write
AI Tier-2 computation (simple model)	<200ms	<1s	Plugin compute + WritebackService write
AI Tier-2 computation (LLM)	<1s	<5s	LLM inference; async always

43. Implementation Checklist

The following checklist covers all six phases of Domain OS implementation. All items marked **MUST** are required for platform integration eligibility.

Phase 1 — Domain Design

Item	Requirement	Check
Root domain concept named and justified	MUST	[]
All domain entities classified: root concept supporting graph-only	MUST	[]
Graph topology designed: all edge types with consent/cycle requirements	MUST	[]
CRDT semantics assigned for every field using decision tree	MUST	[]
AI signals identified: which fields are Tier-2 computed	SHOULD	[]
Space taxonomy defined: at least Personal Team Community	MUST	[]
NamespacePath URI scheme defined: {domain}://{space}/{entity}/{id}/	MUST	[]
Cross-system integration points identified	SHOULD	[]

Phase 2 — Schema Implementation

Item	Requirement	Check
Domain entity structs defined (pure domain language, no storage concerns)	MUST	[]
Error types defined: DomainError, DomainResult<T>	MUST	[]
All Hypercube names follow {domain}.{entity_type} convention	MUST	[]
All primary cubes use N=2 Hybrid encoding as base	MUST	[]
All attribute keys have explicit CRDT semantics assignment	MUST	[]
All attribute keys have explicit PermissionTier assignment	MUST	[]
All AI-computed attributes use PermissionTier::System	MUST	[]
All mandatory attribute keys present (created_at, updated_at, last_actor, etc.)	MUST	[]
DomainStore codec implements bootstrap / write / read / exists	MUST	[]
Round-trip test: read(write(entity)) == entity for all entity types	MUST	[]

Phase 3 — Root Orchestrator

Item	Requirement	Check
Root orchestrator struct owns Grid (or holds Grid reference)	MUST	[]
All domain operations exposed as typed methods (no HyperCell in API)	MUST	[]

Item	Requirement	Check
All mutating methods emit dual-write events (domain log + Hypergrid EventLog)	MUST	[]
hyperql() passthrough exposed	MUST	[]
stats() method exposes domain and Hypergrid substrate metrics	MUST	[]
NamespacePath assigned and registered for all created entities	MUST	[]
Permission check before all mutating operations	MUST	[]
Cycle detection enforced on Dependency edge additions (DFS)	MUST	[]

Phase 4 — Graph and Federation

Item	Requirement	Check
Domain graph wrapper implements: add_edge, remove_edge, reachable_from, detect_cycle	MUST	[]
Hierarchy, Dependency, Association, CrossGridLink edge types supported	MUST	[]
DFS cycle detection tested with 3+ node cycles	MUST	[]
CrossGridLink protocol tested: consent → ShadowCell → sync → writeback	MUST	[]
Federation delta sync tested: offline → reconnect → CRDT convergence verified	SHOULD	[]

Phase 5 — AI Integration

Item	Requirement	Check
At least one health/vitality AI signal defined for root entities	SHOULD	[]
At least one risk or anomaly AI signal defined for root entities	SHOULD	[]
All AI engines registered as HypercubePlugin implementations	MUST (if AI used)	[]
on_cell_mutation() returns correct attr keys for each triggering field	MUST (if AI used)	[]
WritebackService gRPC connection configured and tested	MUST (if AI used)	[]
AI result written as TypedAttrValue::AiSignal with confidence score	MUST (if AI used)	[]
Cache TTL configured per attribute	SHOULD	[]

Phase 6 — Testing and Validation

Item	Requirement	Check
CRDT round-trip: concurrent write from 2 nodes; merge; verify correct result	MUST	[]
OR-Set: concurrent add+add; concurrent add+remove; verify semantics	MUST	[]
Lattice: valid transition; invalid transition; concurrent transitions resolve to LUB	MUST	[]
AS_OF: write at T1; write at T2; AS_OF(T1) returns correct T1 state	MUST	[]
Federation: 2-node cluster; partition; reconnect; verify CRDT convergence	SHOULD	[]
CrossGridLink: full 6-phase protocol test	MUST	[]
Performance benchmarks: HyperRow read <40ms p99; DimSlice <200ms p99	MUST	[]

44. Domain OS Taxonomy — Future Domain Systems

The Domain OS pattern applies broadly. Any domain where the central entity has rich attributes, relationships, time-series history, AI-computable signals, and needs to be federated across organizational boundaries is a candidate for a Domain OS on Hypergrid. The following taxonomy illustrates potential future domain systems, derived by applying the root-concept analysis from Kogi, Ume, and Qala:

Domain OS	Root Domain Concept	Primary Hypercube (example)	Key N=3+ Axes	Critical AI Signals
Healthcare OS (H-OS)	The Patient	healthcare.patients	D ₃ : TimeAxis (visit history)	risk_score, readmission_probabili medication_interaction_flag
Logistics OS (L-OS)	The Shipment	logistics.shipments	D ₃ : TimeAxis (transit states)	delivery_prediction, anomaly_flag route_optimization_score
Legal Practice OS (LP-OS)	The Case	legal.cases	D ₃ : CategoryAxis (jurisdiction)	outcome_prediction, deadline_risk_flag, compliance_score
Research OS (R-OS)	The Study	research.studies	D ₃ : TimeAxis (study timeline)	replication_risk_score, publication_readiness_score
Real Estate OS (RE-OS)	The Property	realestate.properties	D ₃ : TimeAxis, D ₄ : GeoAxis	valuation_score, market_trend_prediction, maintenance_risk_flag
Financial Portfolio OS (FP-OS)	The Position	portfolio.positions	D ₃ : TimeAxis (price series)	risk_adjusted_return, concentration_risk_score, rebalance_signal
Education OS (E-OS)	The Learner	education.learners	D ₃ : TimeAxis (progress)	completion_probability, engagement_score, intervention_flag
Supply Chain OS (SC-OS)	The Supplier	supply_chain.suppliers	D ₃ : TimeAxis, D ₄ : CategoryAxis	supply_risk_score, lead_time_prediction, disruption_flag

Every entry in this table follows the same architectural contract: one root concept, one primary N=2 Hypercube, domain-specific extension axes, mandatory attribute keys, CRDT-assigned fields, and AI signals written via the same WritebackService protocol. This is what it means to be a domain OS on Hypergrid rather than a standalone application.

Appendix A — CRDT Decision Tree (Complete Reference)

CRDT Semantics	When to Use	When NOT to Use	Kogi Example	Ume Example	Qala Example
LastWriteWins (LWW)	Any single-valued field where the latest write is always correct. The default for scalars.	Set-valued fields; lifecycle states; transactional counters; audit trails.	name, due_date, description	module_name, location, fiscal_year	solution_name, version_string, repository_url
OR-Set	Any collection field where concurrent additions from different users/nodes must BOTH survive the merge.	Single-value fields; ordered sequences where position matters.	owners, tags, children, dependencies, links	team_members, risk_owners, okr_contributors	approver_ids, contributor_ids, domain_package_ids
Lattice	Lifecycle and state machine fields that should only advance (no backward transitions without Admin). Must define a partial order over states.	Transient states that may legitimately go backward; any field that is not a finite state machine.	ComponentStatus (Draft→Active→Completed)	ModuleLifecycle (Registered→Active→Archived)	SolutionLifecycle (Draft→InReview→Approved→Active)
PNCounter	Transactional accumulators representing quantities that can both increase and decrease but represent a real accumulated value.	Fields that are not quantities; boolean flags; enum values.	budget_spent, hours_logged	expense_total, headcount	build_count, deployment_count
GrowOnly Counter	Monotonically increasing metrics where decrement is semantically impossible.	Any counter that might legitimately decrease.	view_count, follower_count, click_count	module_restart_count, login_count	ccr_approval_count, build_artifact_count
MaxRegister	Version and sequence numbers that should always reflect the highest observed value.	Fields where 'maximum' is not the correct merge policy.	version (semantic version string)	schema_version, api_version	sde_version, release_build_number
AppendLog	Ordered, causally-consistent audit trails, comment threads, or decision logs.	Structured data requiring per-field updates; large blobs.	activity_log, comment_thread	audit_trail, decision_log	review_comments, change_history
DeepMergeJSON	JSON configuration objects where concurrent partial-key updates should both survive.	Arrays within JSON (use OR-Set instead); data where entire-blob replacement is correct.	plugin_configs, feature_flags	governance_config, notification_config	domain_package_config, quality_gate_config

Appendix B — Mandatory Attribute Keys with CRDT Quick Reference

Attribute Key	CRDT	PermissionTier	JSON Type	Notes
created_at	LWW	System	ISO 8601 string	Set once at creation; never mutated after
updated_at	LWW	System	ISO 8601 string	Updated on every write by domain orchestrator
last_actor	LWW	System	string	Format: {tenant_id}:{identity_tag}:{node_id}
vector_clock	LWW	System	{node_id: u64}	Snapshot of VectorClock at last write; required for federation merge
version	MaxRegister	Editor	semver string	Always takes highest value; never decrements
status	LWW → Lattice v2	Editor	enum string	Current lifecycle status; v2 upgrade converts to Lattice
visibility	LWW	Owner	enum string	Private Protected Public
owners	OR-Set	Owner	[uuid]	Primary ownership list; concurrent adds both survive
tags	OR-Set	Contributor	[string]	Freeform tag set; concurrent adds both survive
policy_ids	LWW	Admin	[uuid]	Governance policy attachments; Admin-only write
namespace_path	LWW	System	URI string	Format: {domain}/{space}/{type}/{id}/; registered in NamespaceRegistry

Appendix C — Error Reference

Error Code	Error	Description	Action
HG-401	PermissionDenied	caller.tier < attr.write_permission	Fix permissions; do not retry
HG-404	NotFound	Entity, cube, or attribute key not found	Verify IDs; do not retry
HG-409	AlreadyExists	Cube or attr key already registered	Idempotent — no-op; safe to ignore
HG-409C	CrdtConflict	LWW losing write (non-fatal, recorded)	Recorded only; no action needed
HG-422G	GrowOnlyViolation	Attempted decrement of GrowOnly counter	Wrong CRDT type; fix code
HG-422L	InvalidStateTransition	Lattice transition not in partial order	Invalid request; fix state machine

Error Code	Error	Description	Action
HG-422C	CyclicDependency	Dependency edge would create cycle	Restructure graph; do not retry
HG-503	StorageError	PostgreSQL / Redis / Kafka unreachable	Retry with exponential backoff
HG-503F	FederationError	Federation peer unreachable	Retry; operate in standalone mode
HG-500S	SchemaConflict	Destructive schema change without migration gate	Fix schema evolution procedure
TT-416	BeyondHotWindow	AS_OF target older than hot EventLog window	Restore from S3 archive and retry

End of Document — Apapo Platform: Domain Operating Systems on Hypergrid

Apapo · Hypergrid NDSS · Kogi IW-OS · Ume B-OS · Qala SF-OS · Version 1.0 · March 2026 · Confidential — Internal Use Only